

Qué es un agente y uso de herramientas

MMIA 6013 — IA Generativa y Agentes · Sesión 11 · lunes 28 de septiembre de 2026

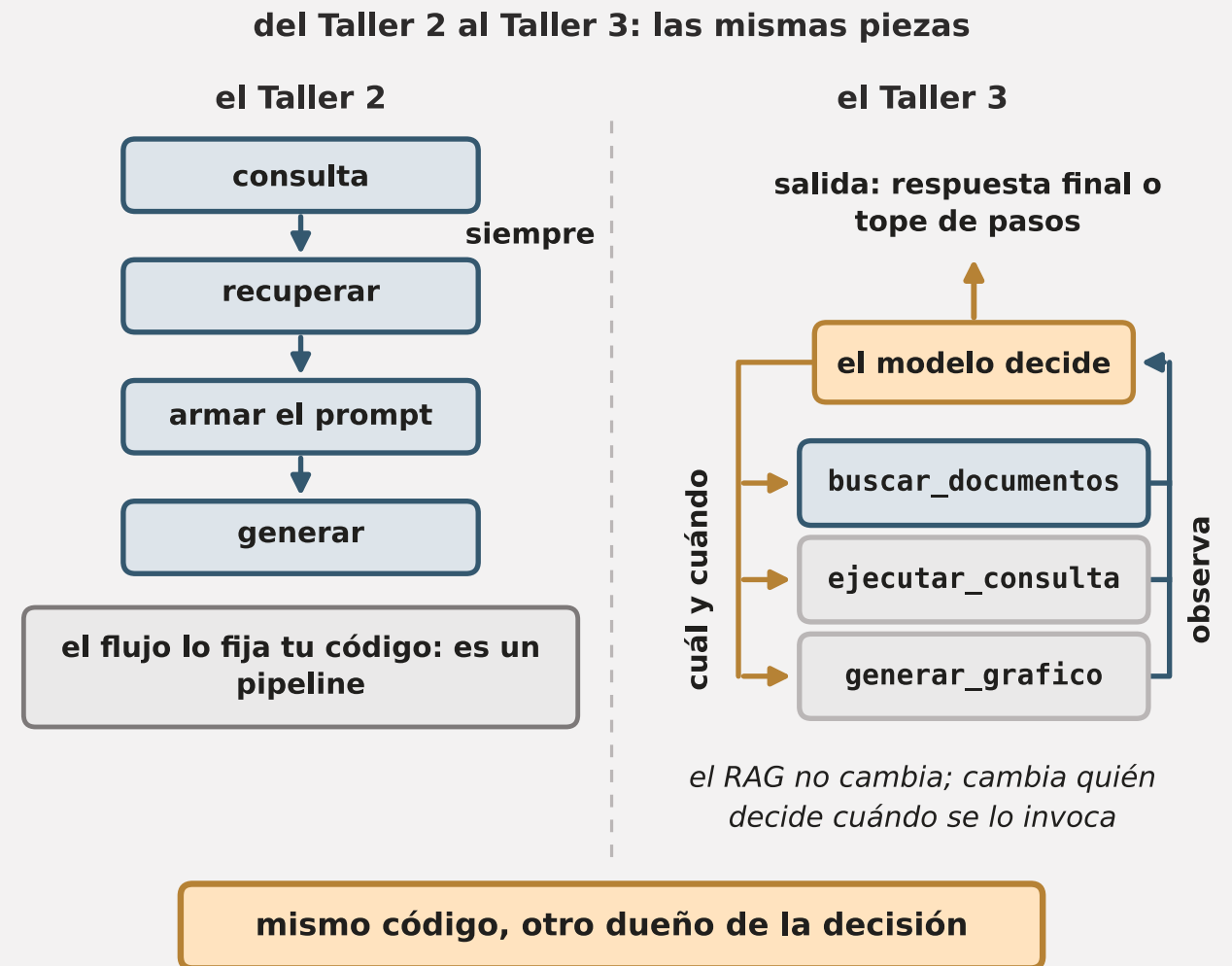
Semana 3 · RA-4 · de los 110 minutos, 50 son el notebook y 60 son este deck

Primera sesión de la semana de agentes. El Taller 3 es el sábado 3 de octubre

Daniel Andrés Riofrío Almeida

El sistema del sábado no es un agente. Hoy se vuelve herramienta

- Cierre literal del Taller 2: un pipeline RAG como el de ese día no es un agente
- No decide qué herramienta invocar ni cuándo: recupera siempre, en el mismo punto
- El sílabo lo pide así: los talleres son acumulativos y ese RAG alimenta al agente del Taller 3
- Hoy no se reescribe una línea del RAG: cambia quién decide cuándo se lo invoca
- Y la sesión 01 dejó una fila pendiente: agente clásico con PEAS → agente que decide
- Las dos promesas se cobran hoy, y en ese orden

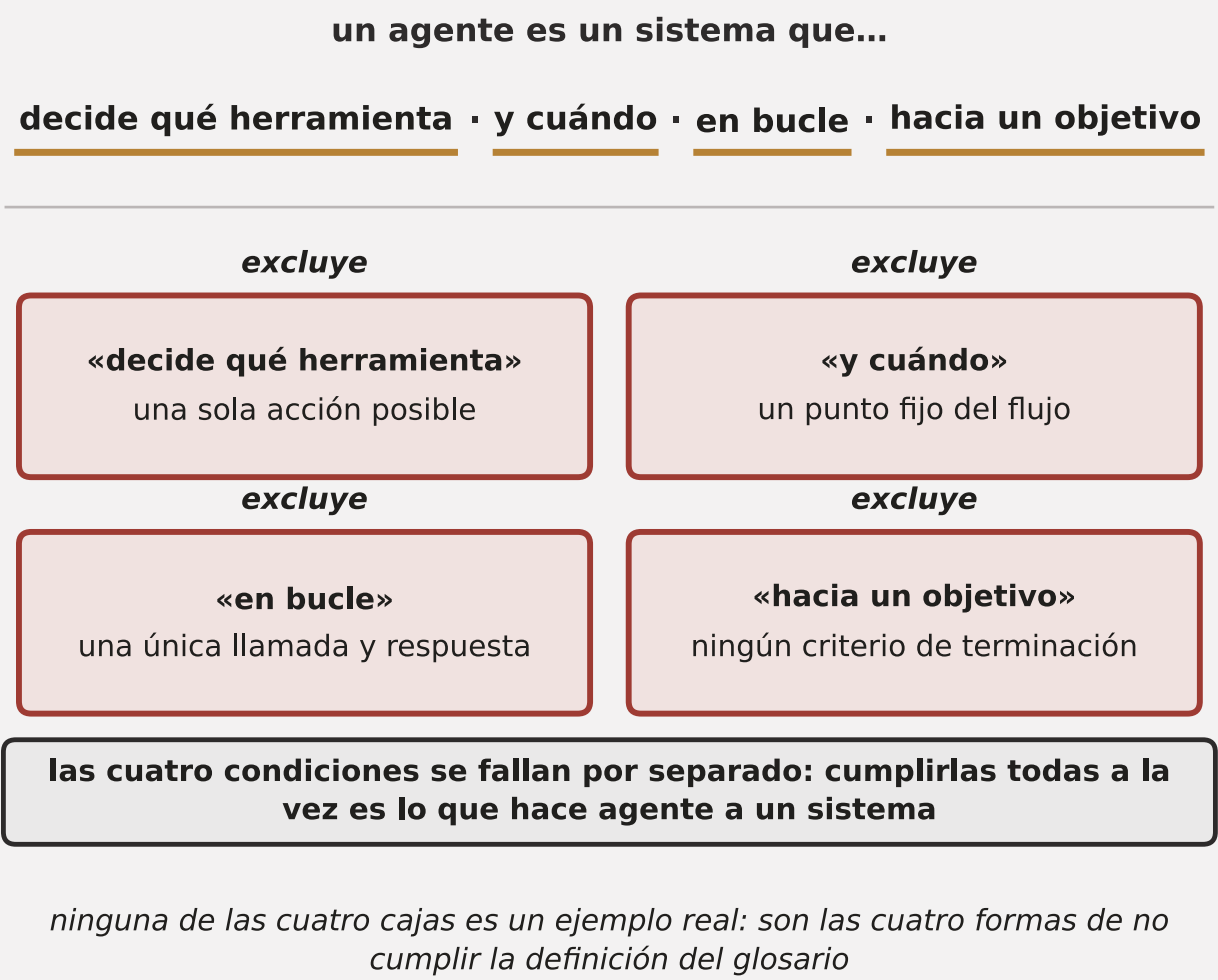


Agente, chatbot, pipeline: quién decide qué

11

La definición del curso, y las cuatro cargas que lleva

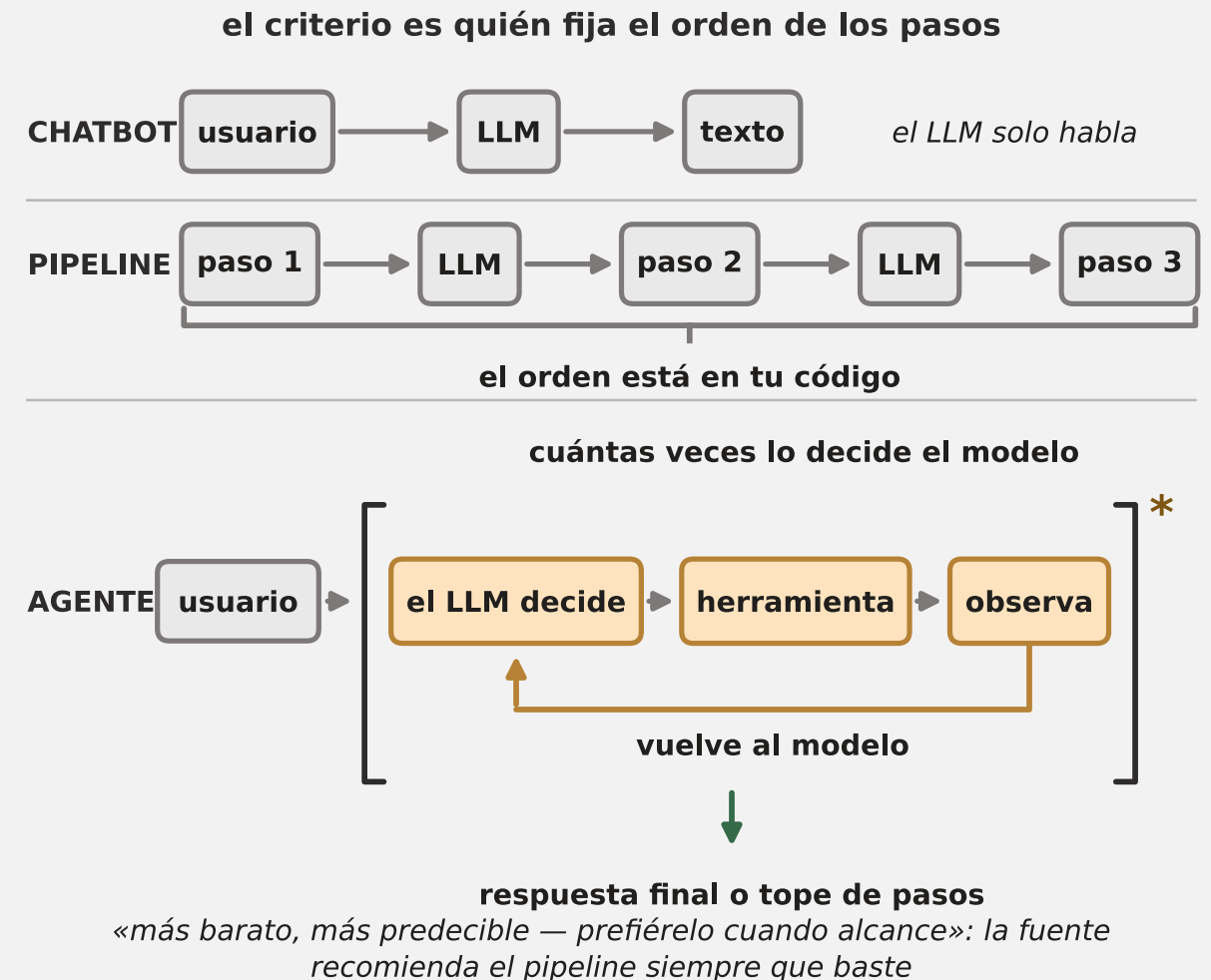
- Agente: sistema que decide qué herramienta invocar y cuándo, en bucle, hacia un objetivo
- «decide qué»: un sistema con una sola acción posible no decide nada
- «y cuándo»: si siempre invoca en el mismo punto, es un flujo fijo
- «en bucle»: una llamada y su respuesta no son un bucle
- «hacia un objetivo»: da la salida por éxito. La salida por fracaso hay que ponerla (al final de la sesión)
- Es la redacción del glosario y no cambia entre diapositivas ni entre sesiones



Fuente: proceso/contexto/glosario.md, fila Agent, que fija la sesión 11 como punto de introducción; fuentes/web/material-divulgacion.md, ítem 3, líneas 68–70, que repite esa frase y la llama «la definición operativa que fija el curso». La lectura pieza por pieza es [AGREGADO] sobre esa misma frase; Alammar & Grootendorst (2024), cap. 7, §«Agents: Creating a System of LLMs», p. 218, que define el agente como el sistema que determina qué acciones tomar y en qué orden

Chatbot, pipeline, agente: el criterio no es la sofisticación

- Chatbot: usuario → LLM → texto. El LLM solo habla
- Pipeline o cadena (chain): paso 1 → LLM → paso 2 → LLM → paso 3. El orden lo fija tu código
- Agente: usuario → [el LLM decide → herramienta → observa]* → respuesta
- Criterio literal: si el flujo de pasos está fijado por tu código, es un pipeline
- Y la fuente prefiere el pipeline cuando alcanza: más barato y más predecible
- La cadena sigue pasos que fijaste tú; el agente determina qué acciones toma y en qué orden



Fuente: curso/actividades/semana-3/s3-lun-explicacion.html, líneas 51–55 (los tres esquemas, redibujados y no copiados) y 57–61 (el criterio y la preferencia por el pipeline, literales); [proceso/contexto/rubrica_calidad.md](#), «Errores graves del dominio»: llamar agente a un chatbot o a una cadena fija de prompts sin decisión sobre herramientas; Alammar & Grootendorst (2024), cap. 7, §«Chains: Extending the Capabilities of LLMs», pp. 202–209 (la cadena secuencial, con su ejemplo de tres llamadas en orden fijo) frente a §«Agents: Creating a System of LLMs», p. 218

¿En qué momento su RAG del sábado pasa a ser un agente?

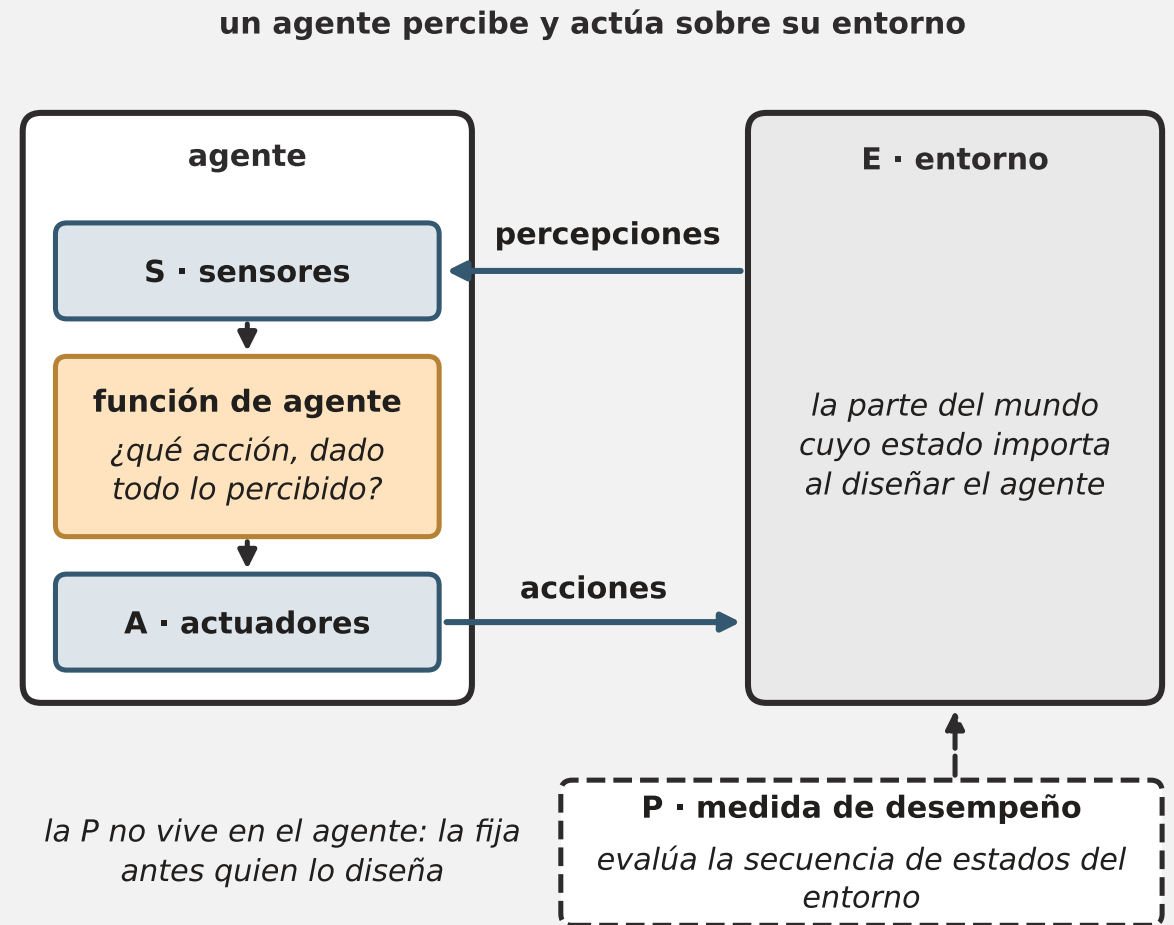
- A. Cuando le añaden la generación con un LLM: ya hay un modelo tomando decisiones
 - *B. Cuando recuperar deja de ser obligatorio y pasa a ser una opción que el modelo elige
 - C. Cuando le conectan una segunda herramienta
 - D. Nunca: recuperar y generar es un pipeline por definición
- Discutan qué cambia en el código en cada caso, no cuál etiqueta suena mejor

El puente con el agente clásico

11

El agente clásico, en cuatro letras

- **PEAS** = *Performance, Environment, Actuators, Sensors*. Los cuatro juntos son el **entorno de tarea**, y especificarlo «tan completamente como sea posible» es el primer paso del diseño
- Un **agente racional** «para cada secuencia de percepciones, selecciona la acción que se espera maximice su medida de desempeño»: la medida viene **antes** del agente, no después
- La función de agente lleva de percepciones a acciones. **Cuatro tipos**, de menos a más: reactivo simple (solo la percepción actual), basado en modelos (guarda estado del mundo), basado en objetivos (elige lo que lo acerca a una meta) y basado en utilidad (compara metas rivales)
- Un agente LLM con herramientas es el **tercero**: persigue el objetivo del prompt. No pondera objetivos rivales, y eso no es un detalle — es de dónde vendrán los problemas del jueves
- Es la reflexión 1 del Taller 3: comparar el agente clásico PEAS con el agente LLM



Fuente: Russell & Norvig (2021), *Artificial Intelligence: A Modern Approach*, 4.^a ed. global, cap. 2 — copia local desde el 2026-09-25, [fuentes/libros/Stuart_J._Russell_Peter_Norvig_-_Artificial_Intelligence__A_Modern_Approach_Global_Edition-Pearson_2021.pdf](#): agente, sensores y actuadores, §2.1, p. 54, y fig. 2.1, p. 55, redibujada; la medida de desempeño evalúa la secuencia de estados del entorno, §2.2.1, p. 57; definición de agente racional, p. 58; PEAS y entorno de tarea, p. 60 y fig. 2.5, p. 62; los cuatro tipos de programa de agente, §2.4.1, p. 67, desarrollados en §§2.4.2–2.4.5, pp. 67–73. Concuerdan fuentes/notas-teoricas/s3-agentes.md §1.1, líneas 9–20, y §1.2, línea 24 (un agente LLM moderno es esencialmente un agente basado en objetivos); curso/talleres/taller-03-agentes.md, línea 100

Los cuatro tipos, y qué añade cada uno

- **Reactivo simple** — una regla sobre la percepción actual. Ignora la historia, así que solo funciona si lo ve todo
- **Basado en modelos** — le añade **estado interno**: sabe lo que dejó de ver
- **Basado en objetivos** — le añade una **meta**. Cambiar la meta no obliga a reescribir las reglas
- **Basado en utilidad** — le añade una **función de utilidad**: pondera metas que se estorban
- El agente LLM con herramientas es el **tercero**, y el prompt es su meta

cada tipo añade una pieza al anterior

tipo	qué añade	qué le permite
reactivo simple	una regla sobre la percepción	ignora la historia
basado en modelos	+ estado interno del mundo	sabe lo que dejó de ver
basado en objetivos	+ una meta que perseguir	cambiar la meta no toca las reglas
basado en utilidad	+ función de utilidad	pondera metas que se estorban

le falta la cuarta fila: sin función de utilidad no sabe elegir entre dos objetivos que se estorban

Fuente: Russell & Norvig (2021), 4.ª ed. global, §2.4.1, p. 67 (la enumeración de los cuatro tipos) y §§2.4.2–2.4.5, pp. 67–73: reactivo simple, p. 67 («seleccionan acciones sobre la base de la percepción actual, ignorando el resto del historial de percepciones»); basado en modelos, fig. 2.12, p. 71; basado en objetivos, fig. 2.13, p. 72; basado en utilidad, §2.4.5, p. 72 y fig. 2.14, p. 73. Los agentes que aprenden, §2.4.6, p. 74. La ubicación del agente LLM como basado en objetivos sale de fuentes/notas-teoricas/s3-agentes.md §1.2, línea 24

El marco no cambia; cambia dónde vive la función de agente

- A · actuadores: las llamadas a herramientas y la respuesta al usuario
- S · sensores: la entrada del usuario y los resultados de las herramientas
- P · desempeño: ¿resolvió la tarea? Medirlo es la semana 4
- E · entorno: lo que sus herramientas leen o modifican. El marco es de Russell; **el mapeo a este curso lo propongo yo**
- El agente clásico tiene su función de agente escrita en código; el agente LLM, en sus pesos
- Su entorno es **parcialmente observable**: el modelo solo percibe lo que va en el prompt
- **⚠** Y el modelo **no tiene estado**. Russell: un entorno totalmente observable es cómodo porque el agente «no necesita mantener estado interno». El suyo no lo es, así que ese estado lo mantiene **su código**, reenviando el historial

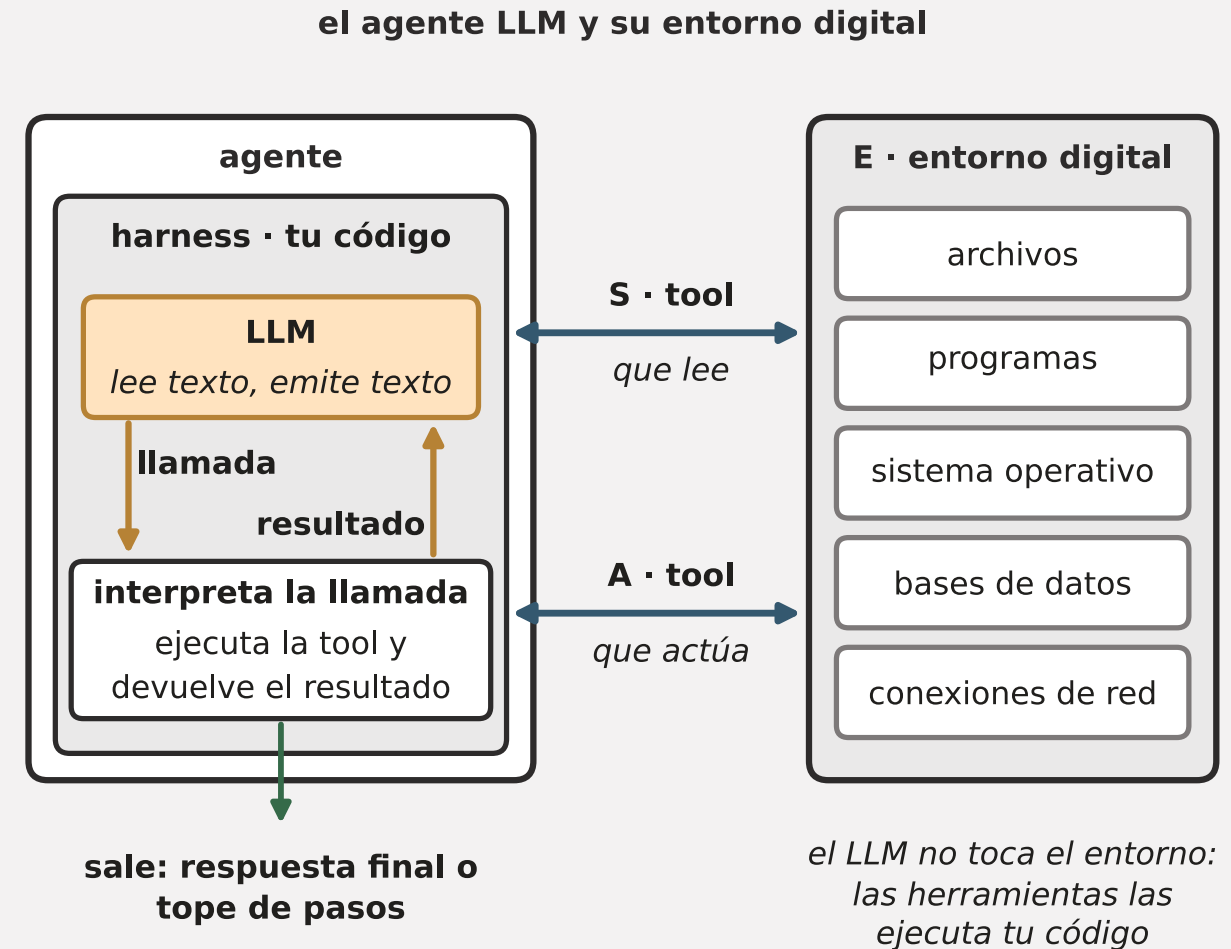
el marco clásico se llena sin problema		
letra de PEAS	agente clásico	agente LLM
P · medida de desempeño	¿alcanzó su meta?	¿resolvió la tarea? — se mide en la semana 4
E · entorno	el mundo en el que actúa (R&N p. 60)	los sistemas que sus herramientas leen o modifican [mapeo propuesto]
A · actuadores	efectores	llamadas a herramientas y respuesta al usuario
S · sensores	percepciones	entrada del usuario y resultados de las herramientas
función de agente	código explícito que escribiste tú	está en los pesos del modelo y no la puedes leer

el modelo no tiene estado y no recuerda; lo que parece memoria es tu código reenviando el historial en cada llamada

Fuente: Russell & Norvig (2021), cap. 2 —copia local desde el 2026-09-25— para el marco: entorno de tarea, p. 60; propiedades del entorno (observable, determinista, episódico o secuencial, conocido) §2.3.2, pp. 62–64; «los entornos totalmente observables son cómodos porque el agente no necesita mantener ningún estado interno», p. 62. Y fuentes/notas-teoricas/s3-agentes.md §1.2, tabla líneas 26–32 y línea 34; curso/actividades/semana-3/s3-lun-explicacion.html §1, tabla líneas 43–48. La casilla E ya no es un agregado sin respaldo —Russell la define—, pero **qué es el entorno en ESTE curso sigue siendo una propuesta del profesor**, derivada de la actividad de hoy y de Lab-03. La corrección sobre «memoria» sale de Alammar & Grootendorst (2024), cap. 7, §«Memory: Helping LLMs to Remember Conversations», p. 209 («these models are stateless — they have no memory of any previous conversation») y §«Conversation Buffer», p. 210 (la memoria se construye copiando el historial dentro del prompt); concuerdan proceso/contexto/rubrica_calidad.md y taller-03, línea 83

El mismo marco, en digital: el agente LLM

- El **entorno** es digital: archivos, programas, el sistema operativo, bases de datos, conexiones de red
- Sensores y actuadores son **herramientas**: una que lee sirve para percibir; una que escribe o ejecuta, para actuar
- Cada uso es una **llamada en dos sentidos**: sale la petición y vuelve el resultado
- Dentro del agente, el LLM solo lee y emite texto. Lo envuelve un **harness** —su código— que interpreta ese texto, ejecuta la herramienta y le devuelve el resultado
- Harness y LLM giran en un **lazo** que termina con la respuesta final o con el tope de pasos



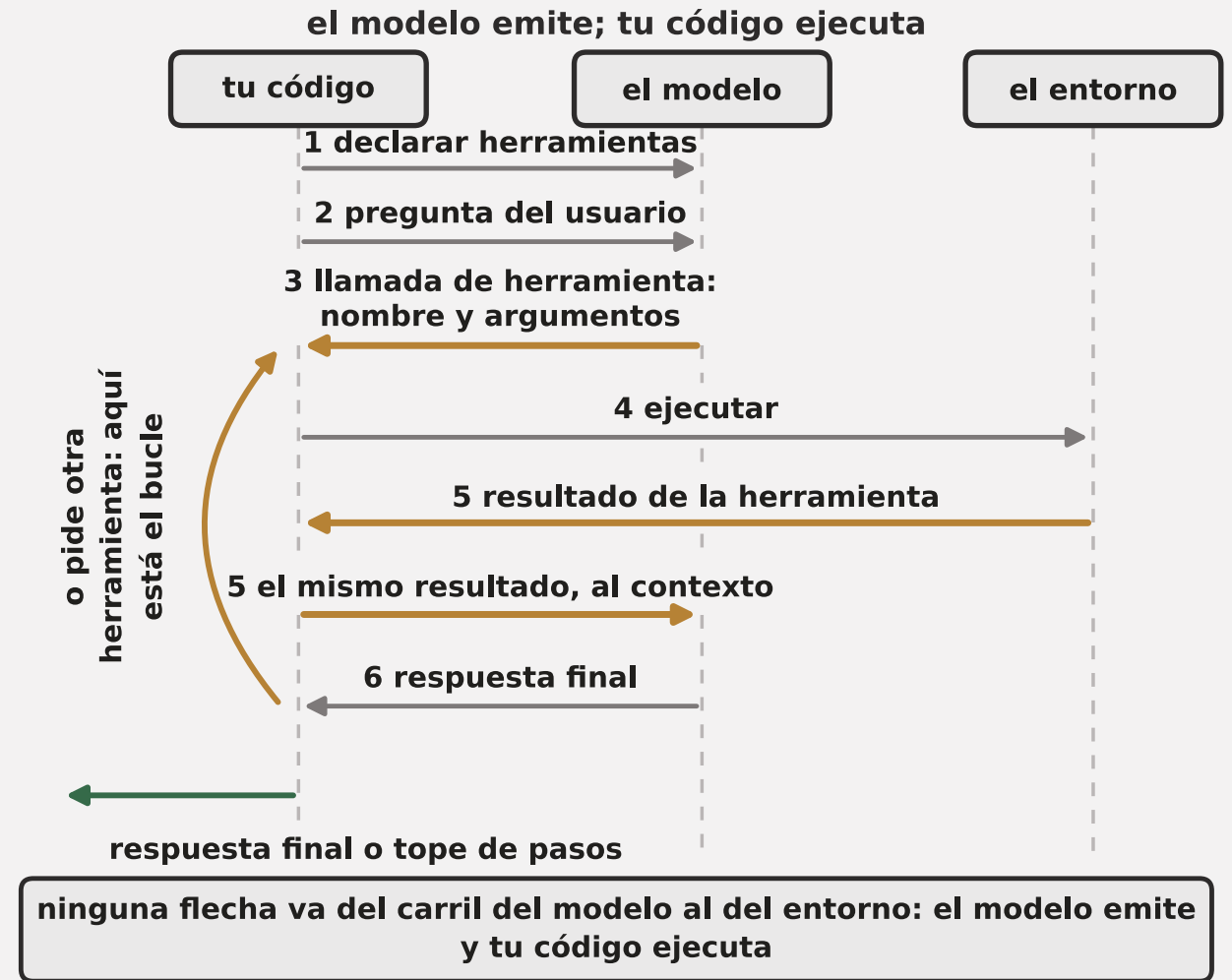
Fuente: Russell & Norvig (2021), 4.^a ed. global, §2.1, p. 54 —el agente de software percibe contenido de archivos, paquetes de red y entrada humana, y actúa escribiendo archivos, enviando paquetes y mostrando información— y fig. 2.1, p. 55, que es la composición de «El agente clásico, en cuatro letras». Alammar & Grootendorst (2024), cap. 7, §«Agents: Creating a System of LLMs», p. 218 (los agentes interactúan con el mundo real mediante herramientas) y §«The Driving Power Behind Agents», p. 219 (el LLM solo puede generar texto). El lazo y su salida: [curso/actividades/semana-3/s3-lun-explicacion.html](#), líneas 68–75, y [fuentes/notas-teoricas/s3-agentes.md](#) §2.1, línea 42. «Harness» es vocabulario de este curso, sin fila en el glosario

El mecanismo: el modelo emite, tu código ejecuta

11

Los seis pasos, y dónde está exactamente el bucle

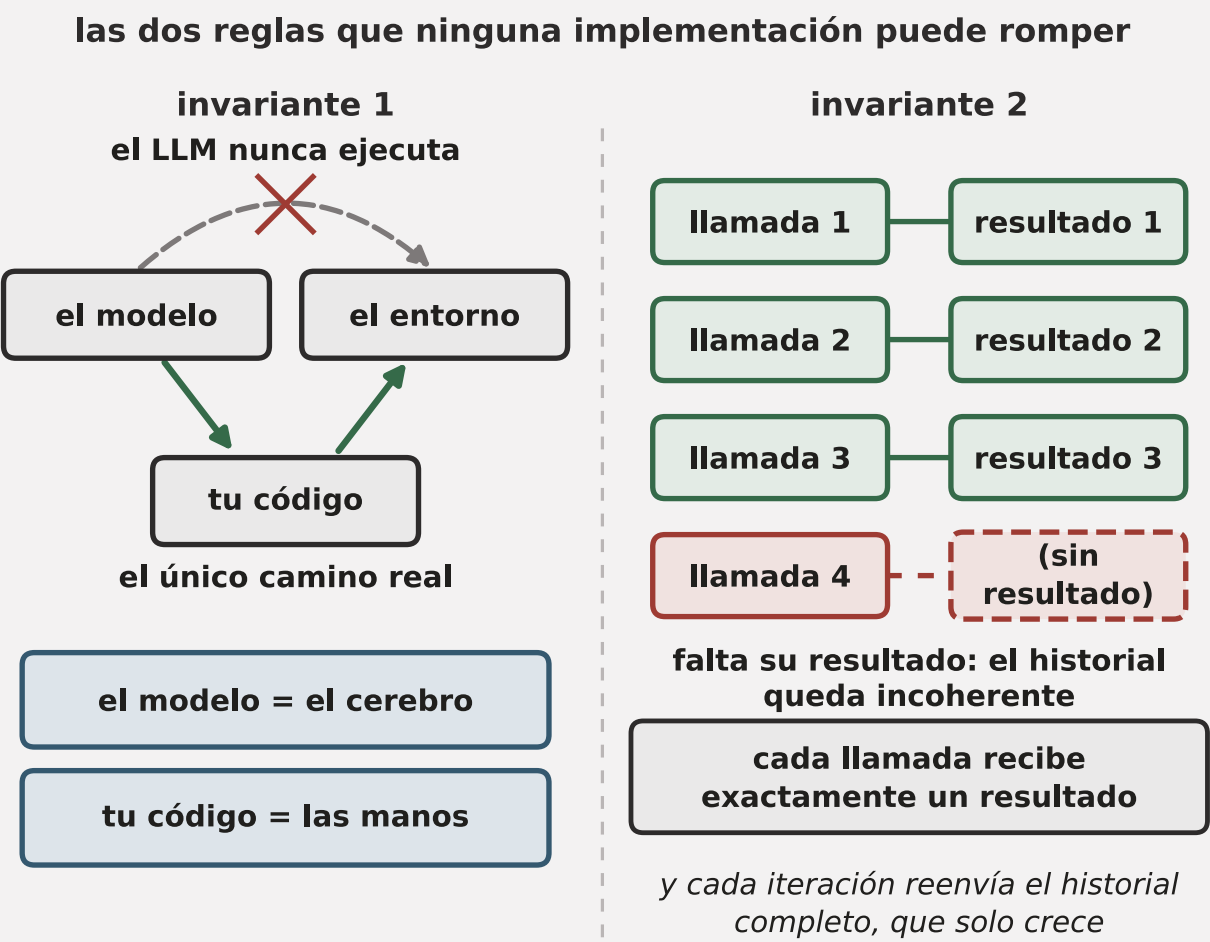
- 1 · Tú declaras las herramientas: nombre, descripción y esquema de parámetros
- 2 · El usuario pregunta
- 3 · El modelo emite la llamada: nombre y argumentos. Solo texto
- 4 · Tu código ejecuta la función con esos argumentos
- 5 · Tú devuelves el resultado, que se añade al contexto
- 6 · El modelo responde — o pide otra herramienta y se vuelve al 3. Eso es el bucle: sin ese retorno hay llamada a función, pero no hay agente



Fuente: [curso/actividades/semana-3/s3-lun-explicacion.html](#), líneas 68–75 (los seis pasos) y 133–134 (por qué el resultado vuelve al modelo y no al usuario); [fuentes/notas-teoricas/s3-agentes.md](#) §2.1, línea 42, y §2.2, líneas 65–73. Ninguna firma de proveedor se proyecta: [fuentes/web/fuentes-web.md](#) no registra fecha de consulta para esta sesión; Alammari & Grootendorst (2024), cap. 7, §«The Driving Power Behind Agents: Step-by-step Reasoning», p. 219 (el modelo solo puede generar texto: la llamada se emite, no se ejecuta)

Las dos invariantes del bucle

- Uno: el LLM nunca ejecuta nada. Solo emite «quiero llamar a X con estos argumentos»
- El modelo es el cerebro; tu código, las manos
- Dos: cada llamada de herramienta recibe exactamente un resultado de herramienta
- Una llamada sin su resultado deja el historial incoherente: es el primer error del sábado
- Consecuencia: cada iteración reenvía el historial entero. Crece hasta topar con la ventana



Fuente: [curso/actividades/semana-3/s3-lun-explicacion.html](#), líneas 64–66 (el LLM nunca ejecuta; el cerebro y las manos); [curso/actividades/_fuentes/s3-lun.py](#), líneas 19–20 (las dos invariantes, nota docente). El crecimiento del historial es Alammar & Grootendorst (2024), cap. 7, §«Windowed Conversation Buffer», p. 212 («as the size of the conversation grows, so does the size of the input prompt until it exceeds the token limit»), y su compromiso está tabulado en la Tabla 7-1, p. 217. No se proyecta ninguna cifra de mercado

«El LLM ejecutó una consulta SQL en mi base de datos.» ¿Qué está mal?

A. Nada: con function calling el modelo ejecuta la consulta que él mismo escribió

*B. La palabra «ejecutó»: el modelo emitió una estructura y otro la ejecutó

C. Nada, siempre que le hayan dado permisos de solo lectura

Discutan qué cambia en el diseño según cuál de las tres sea cierta

La forma del esquema: la estructura que usa el material del curso

- Tres campos: nombre, descripción y esquema de los parámetros
- Los parámetros declaran tipo, formato y —cuando el dominio es cerrado— sus valores
- Los nombres de los campos copian la API de un proveedor, y no se verificó si esa firma sigue vigente: la del sábado sale de la documentación del proveedor que usen

```
CONSULTAR_VENTAS = {  
    "name": "consultar_ventas",  
    "description": ("Detalle de ventas de un mes. Usala cuando "  
                    "pidan el detalle. NO calcula agregados."),  
    "input_schema": {  
        "type": "object",  
        "properties": {"mes": {  
            "type": "string",  
            "description": "Mes YYYY-MM, p.ej. 2026-03"}},  
        "required": ["mes"],  
    },  
}
```


La descripción es el prompt

- Cada herramienta = función Python + esquema. El modelo la elige leyendo su descripción
- Débil: «consulta la base de datos», con un parámetro q de tipo cadena
- Fuerte: empieza con un verbo, dice qué devuelve, y dice cuándo usarla y cuándo NO
- Fuerte: los parámetros llevan formato y dominio, no solo tipo
- Si el agente elige mal, la primera hipótesis son las descripciones, no el modelo
-  El agent.py del Lab 03 no le dice al modelo qué herramientas existen: el sábado lo arreglan

la elección de herramienta se juega en la descripción

débil	fuerte
«consulta la base de datos» parámetro q: string	«ejecuta una consulta de solo lectura sobre la tabla ventas. Úsala cuando pregunten por cifras de ventas. NO sirve para inventario» parámetro mes: string, formato YYYY-MM

✓ verbo y qué devuelve

✓ cuándo sí y cuándo NO

✓ formato y dominio

y el andamiaje que se califica el sábado no tiene ese texto

Lab 03 · SYSTEM_PROMPT	el prompt dice que puede usar herramientas y no nombra ninguna
Lab 03 · TOOL_REGISTRY	el registro asocia 3 nombres a funciones, sin descripciones ni esquemas

Aprender a llamar no es decidir en bucle

11

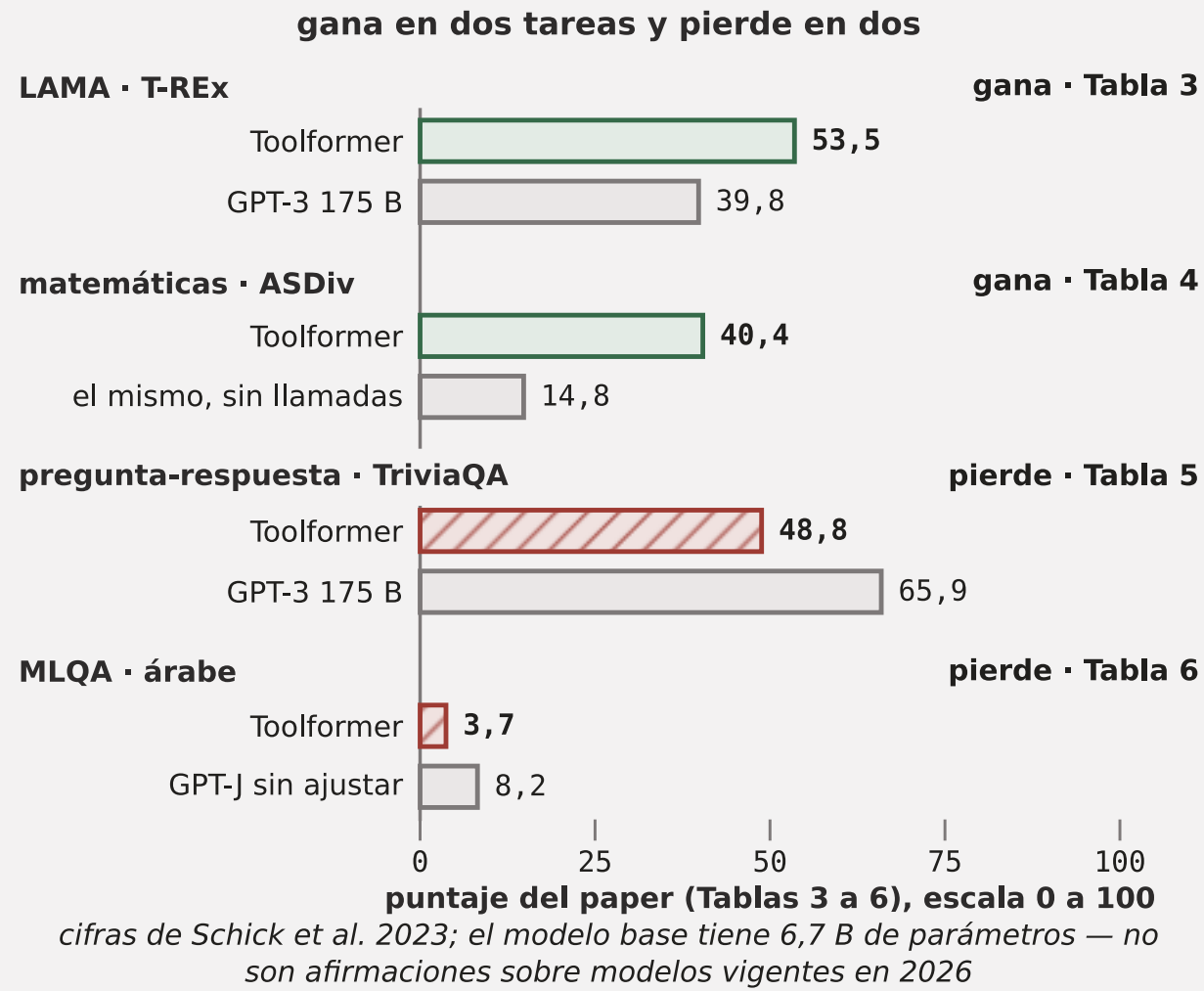
Toolformer: un método de ajuste fino, no un agente

- **Qué es:** un método de ajuste fino (Schick et al., Meta AI, 2023) para que un LLM aprenda a escribir llamadas a APIs dentro de su propio texto: cuál, cuándo y con qué argumentos
- **Por qué aquí:** usa herramientas y, aun así, no es un agente. Es el contraejemplo que deja ver qué pieza falta
- **Cómo aprende:** muestrea llamadas candidatas, las ejecuta y conserva solo las que reducen la pérdida; después se ajusta sobre ese corpus: cambia θ
- La supervisión no es humana: es la propia pérdida de predicción del siguiente token
- Cinco herramientas: QA, buscador BM25 sobre Wikipedia, calculadora, calendario, traductor
- El function calling de hoy no cambia θ : la maquinaria alrededor de la llamada es tu código

Fuente: [fuentes/papers/s3-agentes/schick-2023-toolformer.pdf](#), resumen, p. 1 (Meta AI Research; «a model trained to decide which APIs to call, when to call them, what arguments to pass»), §2, pp. 2–4 (muestrear, ejecutar y filtrar por reducción de la pérdida; ajuste sobre el corpus aumentado) y §3, p. 4 (las cinco herramientas y los dos requisitos para que algo sea herramienta); [proceso/contexto/glosario.md](#), filas Self-supervision y BM25. Ninguna figura del paper se incrusta

Toolformer: dónde gana y dónde pierde

- Base GPT-J de 6,7 B, evaluado en zero-shot, frente a GPT-3 de 175 B
- LAMA —completar un hecho que le falta a una frase corta, como una fecha o un lugar— (SQuAD/Google-RE/T-REx): 33,8 / 11,5 / 53,5 contra 26,8 / 7,0 / 39,8
- Matemáticas: habilitar las llamadas «more than doubles performance» en las tres tareas
- Pero en pregunta-respuesta pierde: 26,3 / 17,7 / 48,8 contra 29,0 / 22,6 / 65,9
- Y en MLQA —párrafo en inglés, pregunta en otro idioma— el árabe pierde contra el GPT-J sin ajustar: 3,7 contra 8,2
- En LAMA se le prohíbe el buscador y en QA se le desactiva el QA: no son sistemas completos



Fuente: [fuentes/papers/s3-agentes/schick-2023-toolformer.pdf](#): §4.1–4.2, p. 5 (GPT-J 6,7 B, zero-shot, los comparativos); Tabla 3 y §4.2.1, p. 6; Tabla 4 y §4.2.2, p. 6; Tabla 5 y §4.2.3, p. 7; Tabla 6 y §4.2.4, p. 7. Cifras del paper de 2023: no son afirmaciones sobre modelos vigentes en 2026

Entonces, ¿Toolformer es un agente?

A favor: elige la herramienta, escribe los argumentos y decide en qué punto llamarla

En contra: como máximo UNA llamada por entrada, «to make sure the model does not get stuck in a loop»

En contra: el resultado se inserta en el flujo de tokens y la decodificación sigue

En contra: §7 declara que no encadena herramientas y que no las usa de forma interactiva

Con la definición de hace media hora: ¿qué le falta — el qué, el cuándo, el bucle o el objetivo?

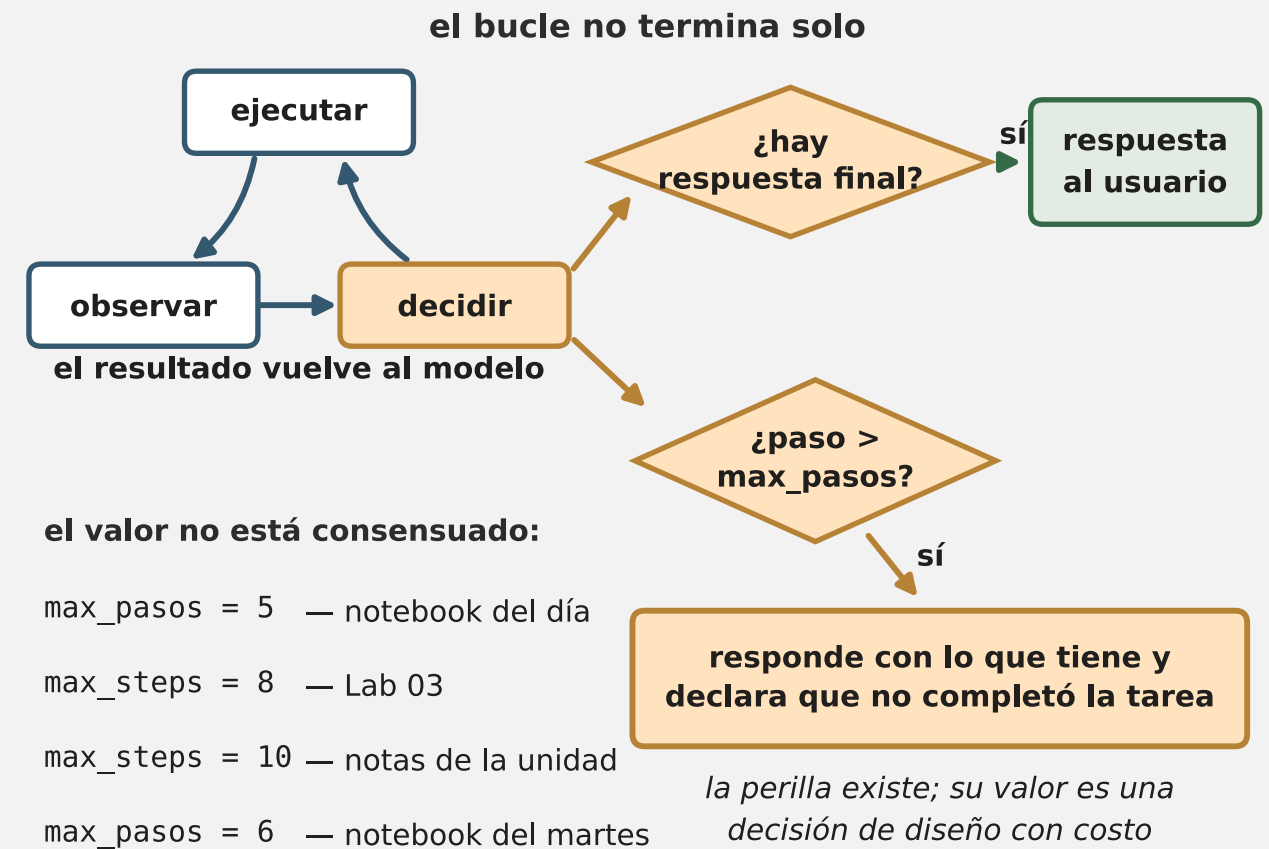
Qué quedó de Toolformer, y qué cambió

- **Por qué importa:** mostró que un LLM puede aprender cuándo llamar a una herramienta y con qué argumentos, sin anotación humana. Los papers de agentes de 2023 ya lo citan como antecedente
- **Lo que quedó:** la llamada es texto que el modelo escribe —nombre y argumentos— y la ejecuta otro programa. Es el mecanismo de los seis pasos de hoy
- **Lo que cambió, 1:** en Toolformer cada herramienta se aprende en los pesos, con ejemplos y reentrenamiento. Hoy se declara en el esquema y θ no se toca
- **Lo que cambió, 2:** una llamada por entrada y sin encadenar. Hoy el harness corre un lazo de varias llamadas, cada una sobre el resultado de la anterior
- **Dónde reaparece:** en la recuperación adaptativa, donde el modelo decide cuándo buscar. Su buscador era un BM25: el RAG de la semana pasada, como herramienta

Fuente: [fuentes/papers/s3-agentes/schick-2023-toolformer.pdf](#), resumen, p. 1 (auto-supervisado, «a handful of demonstrations for each API»), §2, pp. 2–4 (el ajuste sobre el corpus aumentado), §4.2, p. 5 (una llamada por entrada) y §7, p. 11 (no encadena ni interactúa); [fuentes/papers/s3-agentes/shinn-2023-reflexion.pdf](#), §1, p. 1 (lo cita junto a ReAct entre los antecedentes); [fuentes/papers/s3-agentes/yao-2023-react.pdf](#), §2, p. 3 (el lazo pensamiento, acción, observación); [fuentes/papers/s2-rag-vector-search/gao-2023-rag-survey.pdf](#), §V.C «Adaptive Retrieval», p. 11 (lo pone en la tendencia de que el LLM decida cuándo buscar). El esquema de hoy: [fuentes/notas-teoricas/s3-agentes.md](#) §2.1

El límite de pasos no es una optimización: es la condición de parada

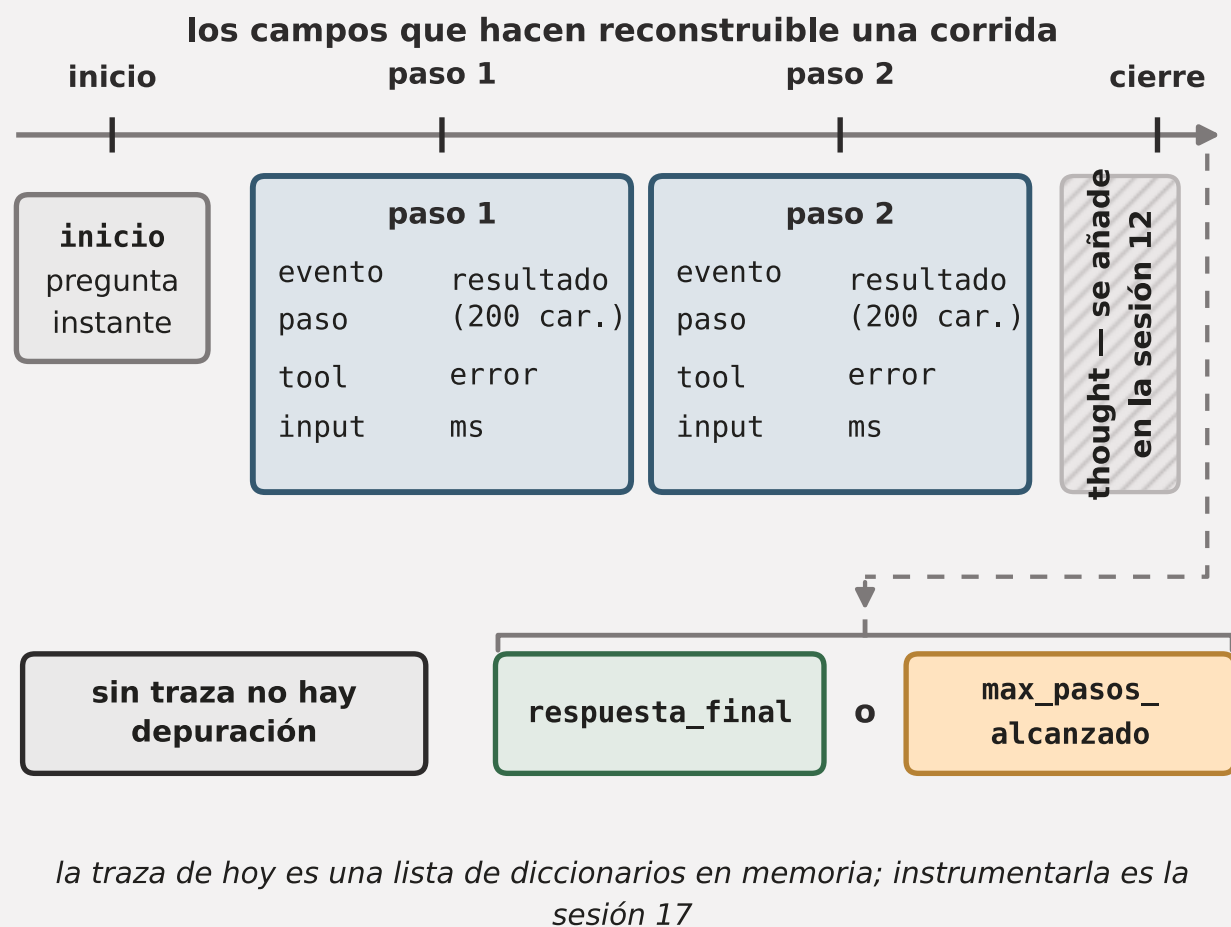
- El bucle decidir → ejecutar → observar no termina solo. Hay que ponerle un tope
- «Todo agente necesita un límite de pasos: es tu seguro anti-loop»
- Al alcanzarlo, el agente responde con lo que tiene y declara que no pudo completar la tarea
- En la misma semana circulan cuatro: 5 y 6 en los notebooks, 8 en el Lab 03, 10 en las notas
- El deck no fija ninguno: la perilla existe y su valor es una decisión de diseño con costo
- El presupuesto de tokens, llamadas y dinero es otra cosa, y es de mañana



el presupuesto de tokens, llamadas y dinero es otra perilla y es de la sesión
12

El notebook, 50 minutos: el bucle y la traza

- Partes 1 a 3: declarar tres herramientas, ejecutar la llamada y escribir el bucle con `max_pasos`
- Corre sin credencial: sin clave en el entorno, un LLM simulado emite las mismas estructuras
- Parte 4, la traza: inicio, y por cada paso evento, tool, input, resultado, error y ms
- Al cierre, `respuesta_final` o `max_pasos_alcanzado`. Sin traza no hay depuración
- La calidad de la traza es un criterio propio del Taller 3: pasos auditables y uso correcto
- El taller la pide con Thought, Action y Observation: ese vocabulario llega mañana



La taxonomía de la sala de juntas

Un mapa de divulgación ordena la IA en cuatro capas: ML · IA generativa · agentes · Agentic AI

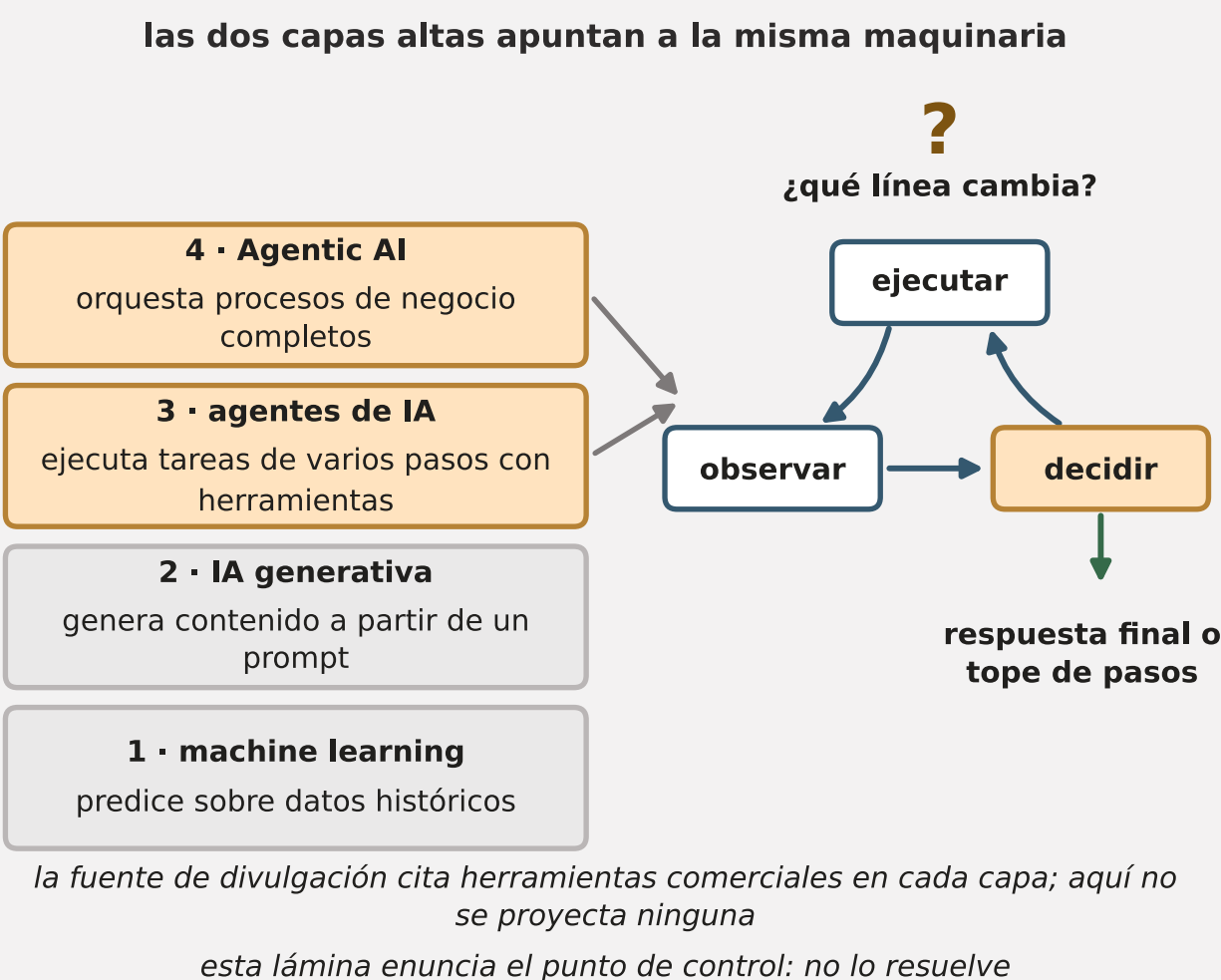
(a) El bucle que acaban de escribir, ¿es de la capa 3 o de la capa 4?

(b) ¿Qué línea de su código cambiaría para pasar de una a la otra?

(c) ¿Qué distingue realmente la 3 de la 4?

(d) ¿Dónde cae un chatbot con un prompt largo y sin herramientas?

Es una afirmación analizada, no una fuente: esta diapositiva pregunta, no dictamina



Mañana los patrones; el sábado, el agente

- El agente = LLM que decide + tu código que ejecuta + bucle con límite de pasos
- La descripción de la herramienta es el prompt que gobierna la elección
- Toda corrida deja traza: sin traza no hay depuración ni observabilidad (semana 4)
- Un agente se define por la decisión, no por la escala
- Un agente son dos piezas: las herramientas (hoy) y el tipo, que planifica (mañana: ReAct y los otros)
- Miércoles (13) MCP · jueves (14) multi-agente y riesgos · sábado: Taller 3, 25 %, 3 de octubre